VoiceOps 2026: ZentroMart AI Customer Support Voice Agent

LLM & Generative AI Capstone Technical Report · Consulting & Analytics Club, IIT Guwahati × Gnani.ai
Simulation Reference Date: 20 October 2026 | **Public GitHub:**
https://github.com/KhushiMaheshwari101/voiceops_2026

GitHub Repository: https://github.com/KhushiMaheshwari101/voiceops_2026

Executive Summary

VoiceOps 2026 addresses the operational challenge of high-volume customer inquiries for **ZentroMart**, a fictional Indian e-commerce enterprise handling groceries, apparel, and electronics. The objective was to build an enterprise-grade, voice-first multimodal support assistant that:

1. Responds to inquiries strictly grounded in internal corporate policies and FAQs, completely eliminating hallucinations.

2. Conducts identity-checked multi-turn conversations, preserving state, enforcing privacy verification gates, and executing real mock tool actions.

3. Operates through a low-latency voice pipeline utilizing **Gnani Speech-to-Text (STT)** and **Text-to-Speech (TTS)** (`timbre-v2.5`), with domain-adapted speech normalization for dates, currencies, and alphanumeric identifiers.

The solution was built sequentially across four foundational steps, evaluated against synthetic datasets containing 1,000 orders, 70 products, 21 knowledge base documents, and 96 practice conversation turns.

Step 1: Baseline LLM Agent (Prompting & Role Boundaries)

1. Architecture & Design

• **System Prompt Design**: In Step 1, a baseline system instruction was established (`SYSTEM_INSTRUCTION_STEP1`). The agent was instructed to act as ZentroMart's customer support representative and introduce itself as an AI assistant in its initial response.

• **Scope & Limitations**: The agent possessed no knowledge-base access or document retrieval mechanisms. It answered customer inquiries solely through zero-shot parametric knowledge and general e-commerce assumptions.

2. Baseline Performance & Failure Modes

• **Accuracy on Practice Questions**: The baseline agent was tested across the 52 standardized practice questions (`practice/questions.csv`).

• **Hallucination Rate**: High (~62% failure rate on policy-specific constraints).

• *Example 1 (Wallet Refund Timeline)*: The baseline hallucinated standard bank timelines ("Refunds take 5-7 business days"), completely failing on ZentroMart's distinct policy where Wallet refunds are processed instantaneously within 2 hours (`REF-2`).

• *Example 2 (Unanswerable Inquiries)*: When asked about non-existent policies (e.g., midnight delivery slots, student discounts), the baseline fabricated realistic-sounding policies rather than acknowledging absence of coverage.

• *Example 3 (Festive Sale Return Window)*: The baseline assumed standard 7-day return periods, unaware of the 30-day extended window for apparel ordered during the festive promotion (`RET-8`).

Step 2: Grounded RAG Architecture (Document Grounding & Precedence)

1. Knowledge Base Ingestion & Chunking

• **Structure**: 21 corporate markdown documents (`kb/*.md`) covering returns, cancellations, shipping, damages, warranties, and FAQs.

• **Section ID Preservation**: Chunks were segmented strictly by markdown section headers (`## [SECTION-ID] Title`). Each chunk retained metadata including `doc_id`, `version`, `effective_from`, `status` (`current` vs `superseded`), and `doc_type`.

• **Precedence Hierarchy Engine**: A core architectural requirement was resolving document conflicts (`KB-00.1`):

$$\text{Current Policy} \succ \text{Agent Guide} \succ \text{Help Article} \succ \text{FAQ} \succ \text{Marketing Page}$$

Superseded documents (e.g., `pol_returns_v3_superseded.md`) were isolated with minimal retrieval weight unless the user explicitly queried delivery dates prior to 1 August 2026.

2. Hybrid Retrieval & Catalog Integration

• **Hybrid Search**: Combines sublinear TF-IDF vectorization with SKU/entity extraction from `products.csv` and `pincodes.csv`. When a query references products like "UrbanThread Slim Fit Jeans" or "SKU ELE-0012", catalog attributes (price, seller, return category, warranty) are dynamically fused into the retrieval prompt.

• **Hallucination Prevention**: Explicit negative constraint prompting enforces:

"If the provided context does not contain the answer, say: 'Sorry, I don't have information about that. I can connect you to a colleague who can help.' Never invent policies, dates, or amounts."

3. Step 1 vs. Step 2 Evaluation Results

Metric	Step 1 (Prompt-Only)	Step 2 (Precedence RAG)	Improvement
Overall Accuracy	38.5%	**88.5%**	**+50.0%**
Gold Evidence Recall	0.0%	**92.3%**	**+92.3%**
Hallucination Prevention (Unanswerable)	14.3%	**100.0%**	**+85.7%**
Strict Date & Policy Compliance	22.0%	**94.2%**	**+72.2%**

Key Takeaway: Grounding transformed the assistant from an unreliable generator into a deterministic policy interpreter. Enforcing document precedence eliminated trap responses (e.g., FAQ generalities overriding specific policy terms).

Step 3: Multi-turn Conversational Agent & Tool Integration

1. Identity Verification Flow (`lookup_order`)

To protect customer privacy, order lookups are strictly gated:

1. **Unverified Inquiry**: When an order ID (e.g., `8978 2837`) is received, the tool is called without verification parameters. It returns `verification_required=True` with no private order details disclosed.

2. **Challenge Turn**: The agent prompts the user for exactly one verification factor: the **last 4 digits of their registered phone number** OR their **6-digit delivery pincode**.

3. **Verification Attempt**: Upon receipt, the agent invokes `lookup_order(order_id, verification_value)`. If validated, the session stores customer details and proceeds. If invalid, the agent permits at most one retry before terminating order disclosure.

2. Calendar & Business Day Engine

• **Reference Simulation Date**: Fixed at **20 October 2026**.

• **Business Day Logic**: Business days are defined as Monday through Saturday, excluding declared warehouse holidays (`2026-08-15`, `2026-10-02`, `2026-10-19`, `2026-11-09`).

• **Return Deadlines**: Return windows operate strictly on calendar days from the confirmed delivery date.

3. Human Escalation & Ticket Generation (`create_ticket`)

The agent monitors conversational turns for defined escalation criteria (`ESC-1`):

• Damaged, defective, or missing item claims valued above ■10,000 (e.g., 43-inch Smart TV).

• Repeated contacts regarding the same issue within 7 days (`contacts_last_7_days >= 2`).

• Safety hazards (battery swelling, fire, sparks).

• Demands for unapproved cash compensation or direct human representative requests.

When triggered, `create_ticket` creates a persistent record with a structured summary (≥ 20 characters), and the agent confirms handover only after receiving `created=True`.

Step 4: VoiceOps Speech Integration (Gnani STT & TTS)

1. Voice Pipeline Architecture

Customer calls are processed end-to-end through a low-latency pipeline:

• **Speech-to-Text**: Utilizing `gnani-vachana` SDK with `GnaniSTTClient`. The system transcribes audio clips using language model `en-IN`, handling accented Indian English and Hinglish seamlessly.

• **Audio Pre-Processing / Text Normalization**: Text passed to TTS is normalized for natural auditory comprehension (`agent_conversation_style.md`):

• Long 8-digit order IDs are spaced into separate digits (`8 9 7 8 2 8 3 7`).

• Currency symbols are expanded (■49 $\rightarrow$ `49 rupees`).

• Markdown artifacts and bullet points are stripped.

• **Text-to-Speech**: Utilizing `GnaniTTSClient` with model `timbre-v2.5` and voice `Kaveri`, synthesizing clear, natural 2-3 sentence audio replies (`reply.wav`).

• **Batch Evaluation Runner (`run.py`)**: Accepts an arbitrary folder of customer audio files, processes each turn sequentially, and exports audio responses alongside structured audit logs in JSON and CSV formats.

What Went Wrong, Edge Cases, and Future Improvements

1. **ASR Entity Distortions**:
   • *Challenge*: Customer speech models occasionally transcribe numeric sequences phonetically (e.g., "ele zero zero one two" or "eight nine seven eight").
   • *Fix Implemented*: Regex normalizers handle both contiguous digits and spaced phonetic digit transcriptions.
2. **Coreference in Multi-turn Dialogues**:
   • *Challenge*: Follow-up inquiries such as *"Aur refund kitne din mein aayega?"* lack explicit order IDs.
   • *Fix Implemented*: Session state retains verified order metadata, items, and active payment modes across turns.
3. **Future Enhancements**:
   • Implementing WebSocket streaming for real-time bidirectional voice interruption (barge-in capability).
   • Adding automated phoneme-level pronunciation guides for vernacular Indian product titles.

Conclusion & Deliverable Checklist

• [x] Full Git repository with clean modular architecture in `src/`
• [x] Tested batch command: `python run.py --audio_dir <folder> --output_dir sample_outputs/`
• [x] `.env.example` with zero secret leakage
• [x] 100% verified dataset checks via `validate_dataset.py`
• [x] Quantitative Step 1 vs Step 2 comparative benchmarks